

FIAP

ChargeGrid Intelligence

São Paulo – 2026

FIAP

Caio Henrique Ferraz da Silva – RM:568992

Enzo Caruso Peter – RM: 570908

Leonardo Figueredo dos Santos – RM: 573653

Leonardo Robert Maulicino – RM: 570329

Lucas Ramos de Sousa – RM: 573901

Matheus Pimenta Martini – RM: 569400

Pablo Renato dos Santos Sobral de Carvalho – RM: 569894

São Paulo – 2026

Sumário

O Que o Sistema Faz:	4
Divisão de Frentes de Trabalho:	4
Estado da Entrega:	5
Esquema de Integração dos Componentes:	5
Fluxo Principal – ciclo completo de uma sessão.....	6
Responsabilidade de Cada Componente	6
Justificativa Técnica das Escolhas:	7
Resultados e Dados Funcionais:	8
Conexão com os Conteúdos das Disciplinas:.....	10
Estado Atual e Próximas Etapas:	11

O Que o Sistema Faz:

O ChargeGrid Intelligence gerencia múltiplos carregadores de veículos elétricos em um mesmo ponto de instalação (shoppings, estacionamentos, postos) controlando a demanda de energia.

A plataforma resolve um problema concreto de infraestrutura: um posto com vários carregadores raramente tem entrada elétrica para operar todos na potência máxima simultaneamente. Ampliar a entrada é caro; deixar carregadores ociosos é desperdício. O sistema distribui dinamicamente a potência disponível entre as sessões ativas, respeitando o limite contratado e o teto individual de cada equipamento.

Sobre essa base operam a tarifação dinâmica, que ajusta o preço por horário, demanda agregada e modo de carregamento, e o faturamento automático ao fim de cada sessão.

Divisão de Frentes de Trabalho:

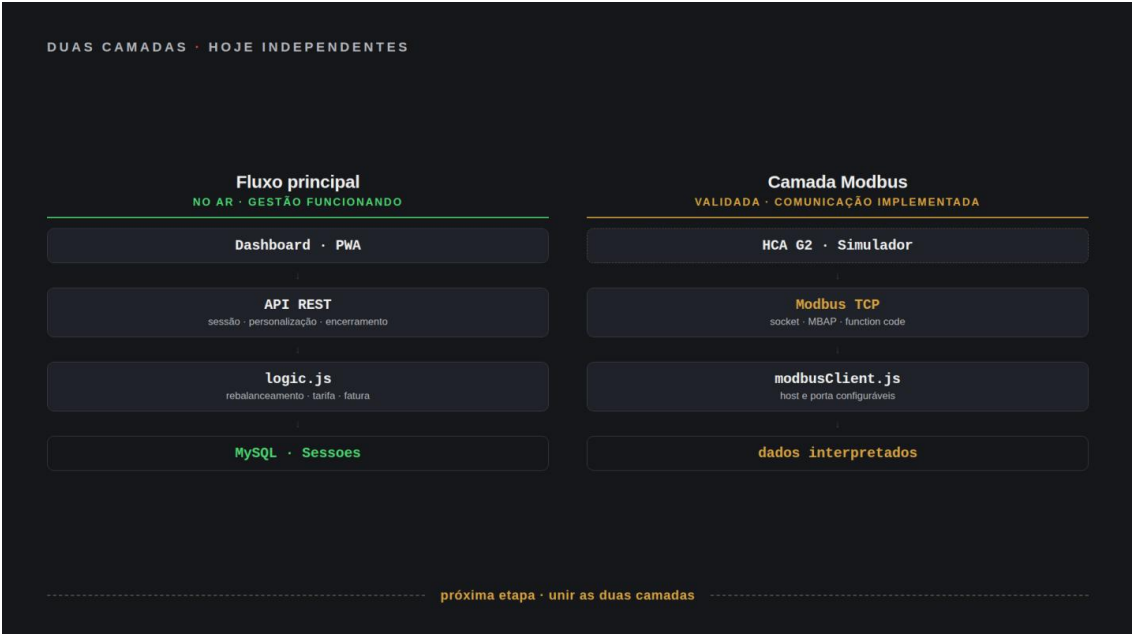
FRENTE	RESPONSABILIDADE	SITUAÇÃO
Backend Foundation	Servidor Node.js + Express + MySQL	Concluído
Banco e regras de negócio	Schema, models e lógica de tarifa e rebalanceamento	Concluído
API REST	Rotas de sessões, carregadores, RFID, usuários e faturas	Concluído
Interface do Operador	Dashboard: estrutura, integração com a API, métricas	Concluído
Interface do Cliente	PWA: login, menu e integração com a API	Em desenvolvimento
Comunicação Modbus TCP	Cliente, simulador de protocolo e mapa de registradores	Validada · integração pendente
Protótipo físico	ESP32 + leitor NFC, comunicando com a API via REST	Em desenvolvimento
Pagamento e faturamento	Fluxo de fatura e tarifação dinâmica	Fatura concluída · gateway externo pendente

Estado da Entrega:

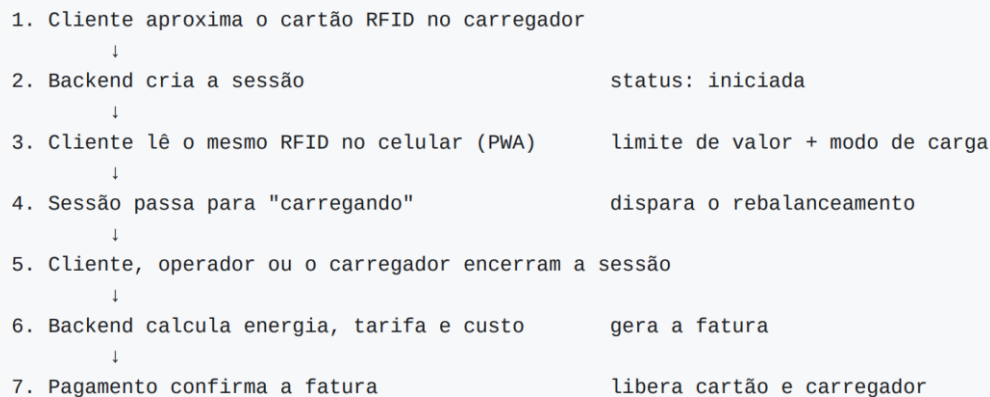
CAMADA	SITUAÇÃO
Gestão de sessões, tarifação e faturamento	Funcional e implantado
Interfaces web (dashboard do operador e PWA do cliente)	Funcionais e publicadas
Comunicação Modbus TCP com o carregador	Implementada e validada em paralelo
Protótipo físico (ESP32 + leitor NFC)	Em desenvolvimento
Integração das duas camadas	Próxima etapa
Validação com carregador HCA G2 físico	Depende de acesso ao equipamento

Esquema de Integração dos Componentes:

O sistema é composto por duas camadas que hoje operam de forma independente. A distinção é deliberada e está refletida na arquitetura.



Fluxo Principal – ciclo completo de uma sessão



As atualizações de estado acontecem por evento — início e encerramento de sessão. Não há laço de consulta contínua no backend: a checagem de consumo em tempo real é feita sob demanda, quando o PWA ou o dashboard solicitam.

Responsabilidade de Cada Componente

COMPONENTE	RESPONSABILIDADE
frontend/tela-operador/	Dashboard: carregadores em tempo real, métricas agregadas, criação e encerramento manual de sessão
frontend/tela-cliente/	PWA instalável: login, vínculo por RFID, personalização da recarga, acompanhamento e pagamento
src/routes/	API REST: sessões, carregadores, cartões, usuários e faturas
src/logic.js	Regra de negócio pura — sem SQL e sem DOM: rebalanceamento, tarifa, energia, limites
src/models/	Acesso ao banco, um model por entidade
MySQL	Persistência de usuários, cartões, carregadores, sessões e faturas
modbusClient.js	Cliente Modbus TCP: conecta, lê registradores e interpreta os valores brutos
modbusSimulator.js	Servidor Modbus TCP que emula o carregador para desenvolvimento e teste
modbusConfig.js	Mapa de registradores e parâmetros de conexão, centralizados
Protótipo físico	ESP32 com leitor NFC, emulando fisicamente o carregador e comunicando com a API via REST — em desenvolvimento

Os três últimos componentes da camada de comunicação (`modbusClient.js` , `modbusSimulator.js` e `modbusConfig.js`) constituem um módulo independente: o `server.js` não os importa, e eles são executados por scripts próprios, sem banco e sem Express.



Justificativa Técnica das Escolhas:

-Carregador e protocolo:

A linha GoodWe HCA G2 comunica via Modbus TCP, um protocolo aberto e amplamente adotado em automação industrial. A escolha tem uma consequência prática direta: a estação não fica presa ao software proprietário de um fabricante, e um posto pode integrar equipamentos de marcas diferentes sob a mesma plataforma de gestão.

O mapa de registradores e os parâmetros de conexão estão isolados em modbusConfig.js. Acrescentar um modelo de carregador ou ajustar endereçamento é uma alteração localizada nessa camada, sem tocar na lógica de negócio.

-Backend em Node.js:

Manter toda a aplicação em uma única linguagem, do cliente Modbus à API REST, reduz o custo de contexto em uma equipe de sete pessoas trabalhando em frentes distintas. A biblioteca modbus-serial cobre TCP e RTU, o que mantém aberta a possibilidade de comunicação serial sem troca de dependência.

-Regra de negócio isolada:

O logic.js não acessa banco nem manipula interface: recebe números e devolve números. A consequência é que o cálculo de tarifa e o algoritmo de rebalanceamento podem ser exercitados isoladamente, sem subir servidor nem

banco de dados, o que tornou possível validar o comportamento do algoritmo com diferentes configurações de carregadores.

-Arquitetura orientada a eventos:

O rebalanceamento é disparado nas transições de estado da sessão, não por consulta periódica. Além de reduzir carga desnecessária sobre o banco, isso garante que a redistribuição de potência aconteça no instante em que ela é de fato necessária, quando um veículo entra ou sai.

-Front-end em JavaScript sem framework:

O PWA precisa ser leve e instalável, com service worker e manifesto. Sem etapa de build, o código servido é exatamente o código versionado, o que simplifica a implantação e a auditoria.

-Simulador que implementa o protocolo:

Em vez de substituir a camada de comunicação por funções de teste, foi construído um servidor Modbus TCP: ele abre socket, expõe registradores e responde com frames Modbus. A diferença é relevante — o mesmo cliente que falará com o equipamento físico é o que se conecta ao simulador. Trocar um pelo outro é mudar endereço e porta.

Decisão de engenharia: O carregador físico ainda não está disponível. Em vez de aguardar o equipamento para começar a integração, a equipe construiu o simulador de protocolo — o que permitiu desenvolver e validar toda a camada de comunicação antecipadamente.

Resultados e Dados Funcionais:

-Rebalanceamento de potência:

O ambiente publicado tem três carregadores cadastrados, de capacidades diferentes: Vaga A (7 kW), Vaga B (11 kW) e Vaga C (22 kW), 40 kW instalados. O limite de demanda é calculado dinamicamente como 80% desse total, resultando em 32 kW operacionais, com margem de segurança sobre a capacidade física.

Com os três carregadores em sessão simultânea, uma divisão igualitária entregaria 10,67 kW a cada um. O problema é que o carregador de 7 kW não absorve essa fatia, 3,67 kW ficariam ociosos. O algoritmo implementado distribui a potência de forma iterativa: atende primeiro quem cabe integralmente dentro da fatia e redistribui a sobra entre os demais.

CARREGADOR	DIVISÃO IGUALITÁRIA	ALGORITMO IMPLEMENTADO
Vaga A · teto de 7 kW	7,00 (limitado)	7,00
Vaga B · teto de 11 kW	10,67	11,00
Vaga C · teto de 22 kW	10,67	14,00
Potência entregue	28,33 kW	32,00 kW
Capacidade ociosa	3,67 kW	0

13% a mais de potência entregue com a mesma infraestrutura instalada, sem ampliar a entrada elétrica do posto. Esse é o resultado central do projeto.

DADOS FUNCIONAIS · RESULTADOS VALIDADOS

CARREGADOR	DIVISÃO IGUAL	WATER-FILLING
Vaga A · 7 kW	7,00 limitado	7
Vaga B · 11 kW	10,67	11
Vaga C · 22 kW	10,67	14
Entregue	28,33 kW	32 kW

Limite seguro: **32 kW** — 80% dos 40 instalados.

Desperdício na divisão igual: **3,67 kW** · no water-filling: **0**

11

cenários Modbus

- ▶ Conector ligado · cartão lido
- ▶ Handshake com o veículo
- ▶ Carregando · alteração de potência
- ▶ Interrupção da carga
- ▶ Carga concluída
- ▶ Alarme do equipamento
- ▶ Perda e retomada de comunicação

-Tarifação dinâmica:

MODO DE CARREGAMENTO	FORA DE PICO	PICO · 18H-21H
Rápido (rede)	R\$ 0,80 /kWh	R\$ 0,96 /kWh
Fotovoltaico com baterias	R\$ 0,50 /kWh	R\$ 0,60 /kWh
Fotovoltaico	R\$ 0,30 /kWh	R\$ 0,36 /kWh

Há ainda um acréscimo de 15% quando a demanda agregada ultrapassa 30 kW ou há três ou mais sessões ativas. O modo fotovoltaico custa 62% menos que o rápido, um incentivo tarifário direto para deslocar o consumo à janela de maior geração solar.

-Camada de comunicação Modbus TCP:

A camada foi exercitada em onze cenários de carregador, contra o simulador de protocolo: ocioso, conector ligado, cartão lido, handshake com o veículo, carregamento, alteração de potência, interrupção por energia solar insuficiente, carga concluída, sessão finalizada, alarme do equipamento, e perda seguida de retomada da comunicação.

Na leitura, os valores brutos dos registradores são convertidos para grandezas do domínio:

REGISTRADOR	VALOR BRUTO	INTERPRETADO
10015 potência	74	7,4 kW (fator de escala 10)
10016 energia	125	12,5 kWh
10017 status	3	Charging in progress → em_uso
10500-10506 cartão	0x4131 0x4232 ...	A1B2C3D4 (ASCII)

O tratamento de falha foi validado explicitamente: quando a leitura em bloco não é atendida, o cliente recorre à leitura registrador a registrador e, persistindo o erro, reporta a exceção Modbus de forma tratada — sem interromper a execução.

Conexão com os Conteúdos das Disciplinas:

-Eficiência energética:

Eficiência, neste projeto, não significa consumir menos, significa não desperdiçar capacidade instalada. É exatamente o que o algoritmo de rebalanceamento faz. Sem ele, atender três veículos simultâneos exigiria ampliar a entrada elétrica do posto; com ele, os mesmos 32 kW contratados atendem os três sem deixar nenhum quilowatt ocioso.

A margem de segurança de 20% sobre a potência instalada pertence ao mesmo princípio: operar abaixo do limite físico protege a infraestrutura e dispensa o sobredimensionamento que seria necessário apenas para absorver picos.

-Automação inteligente:

O sistema mede, decide e age sem operador. O rebalanceamento não é acionado por ninguém: dispara nas transições de sessão, recalcula a distribuição entre todas as sessões ativas e persiste os novos limites. A tarifa é determinada em

tempo de execução a partir do horário, da demanda agregada e do modo escolhido pelo cliente.

A opção por arquitetura orientada a eventos, em lugar de consulta periódica, é uma decisão de eficiência computacional coerente com o princípio: processar apenas quando o estado efetivamente muda.

-Sustentabilidade e integração com energia renovável:

A contribuição do projeto para sustentabilidade não está no veículo ser elétrico, está em quando ele é carregado. A tarifação diferenciada por modo (R\$ 0,30 no fotovoltaico contra R\$ 0,80 no rápido) é um mecanismo de resposta da demanda: em vez de ampliar a geração, desloca-se o consumo para a janela de maior produção solar, elevando a fração de energia renovável efetivamente aproveitada.

Essa lógica é reconhecida pelo próprio equipamento. Entre os estados do HCA G2 há um específico para carga interrompida por energia fotovoltaica insuficiente, o carregador distingue escassez de geração solar de uma falha comum, o que permite ao sistema tratá-la como condição operacional prevista, e não como erro.

-Protocolos abertos e interoperabilidade:

A adoção do Modbus TCP mantém a estação livre de dependência de um único fornecedor de software. Combinada ao isolamento do mapa de registradores em uma camada de configuração, essa escolha permite que o sistema evolua para um parque heterogêneo de carregadores sem reescrita da lógica de gestão.

Estado Atual e Próximas Etapas:

O projeto entrega, nesta sprint, uma plataforma de gestão em operação e uma camada de comunicação com o carregador desenvolvida e validada em paralelo.

-Em operação:

- Ciclo completo de sessão: leitura de cartão, personalização, carregamento, encerramento, fatura e pagamento
- Rebalanceamento de potência entre carregadores de capacidades diferentes
- Limite de demanda calculado dinamicamente a partir da potência instalada
Tarifação dinâmica por horário, demanda e modo de carregamento
- Dashboard do operador e PWA do cliente, ambos publicados
- API REST cobrindo sessões, carregadores, cartões, usuários e faturas

-Desenvolvido e validado, ainda não integrado:

- Cliente Modbus TCP com endereço e porta configuráveis
- Servidor Modbus TCP de simulação, exercitando o mesmo cliente do equipamento real
- Mapa de registradores e interpretação de estados do carregador
- Tratamento de falha de comunicação com degradação controlada

-Próximas etapas:

1. Conectar a camada Modbus ao gerenciamento de sessões e à persistência, substituindo a simulação por eventos via API REST
2. Validar a comunicação com um carregador HCA G2 físico e confirmar os parâmetros do mapa de registradores
3. Integrar gateway de pagamento externo
4. Camada preditiva de antecipação de picos a partir do histórico de sessões
5. Expansão de posto único para múltiplos postos conectados

Sobre o hardware: A validação com o equipamento físico depende de acesso a um carregador da linha HCA G2. Toda a camada de comunicação foi construída para que essa transição seja uma mudança de configuração — endereço, porta e confirmação do mapa de registradores — e não uma reescrita.